

Greedy and Dynamic Programming (Program)

(Interval) Scheduling

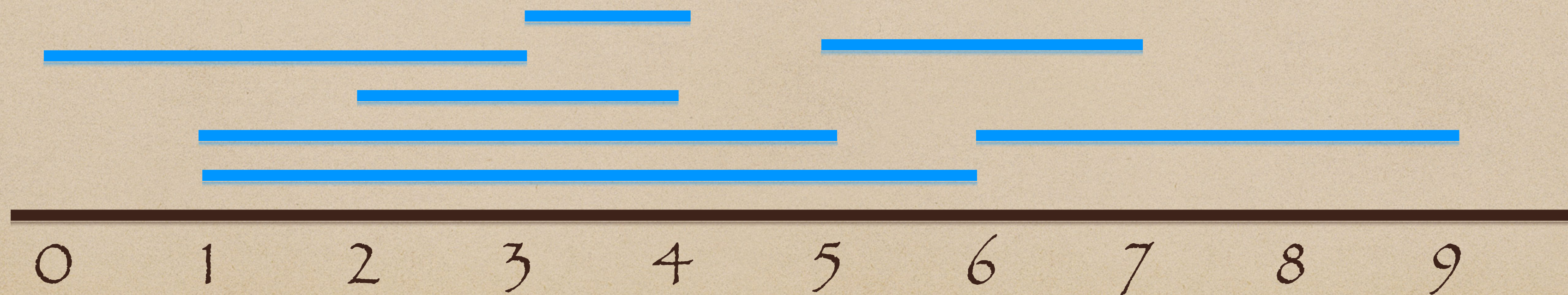
- ◆ We have n requests, where each request r_i has a start time s_i and end time t_i .
- ◆ Each of these requests can be seen as an interval - $[s_i, t_i]$
- ◆ Hence, we have n intervals, $[s_1, t_1], \dots, [s_n, t_n]$.

Goal: Maximize the total number of requests that we serve.

Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

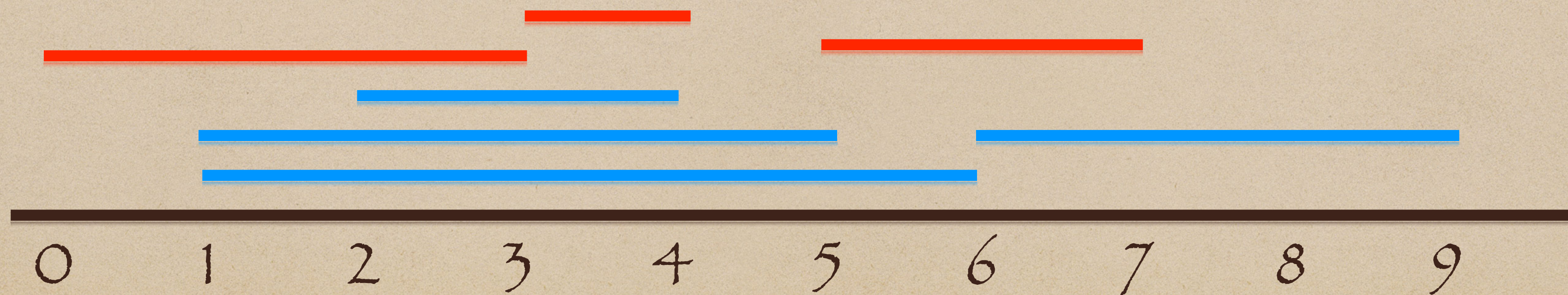
- ◆ Given a set of intervals on a real line, find the **largest** subset of **disjoint** intervals.
- ◆ Ex. $(1, 5)$, $(0, 3)$, $(2, 4)$, $(1, 6)$, $(3, 4)$, $(5, 7)$, $(6, 9)$.



Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

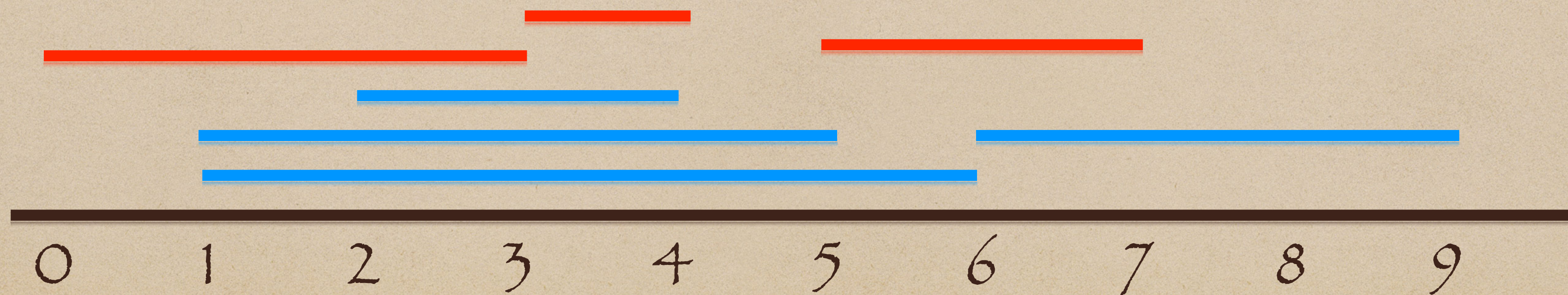
- ◆ Given a set of intervals on a real line, find the **largest** subset of **disjoint** intervals.
- ◆ Ex. $(1, 5)$, $(0, 3)$, $(2, 4)$, $(1, 6)$, $(3, 4)$, $(5, 7)$, $(6, 9)$.



Back to Interval Scheduling

[Kleinberg, Tardos; Chap. 4]

We have designed a Greedy Algorithm which computes an optimum solution in polynomial time.

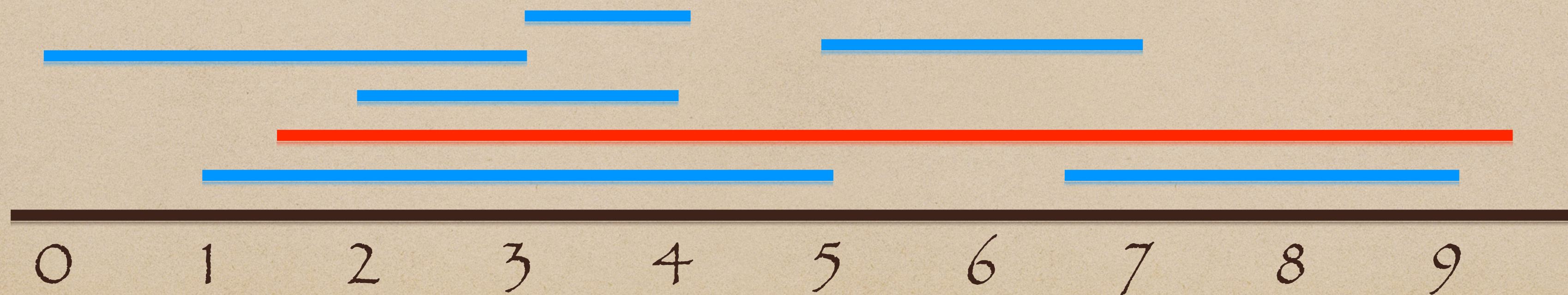


Interval Scheduling - A new Variant

- ◆ Select a set of intervals to maximise the **total length** of the selected intervals.
- ◆ For instance, there could be **profit** proportional to the duration of “use” time.

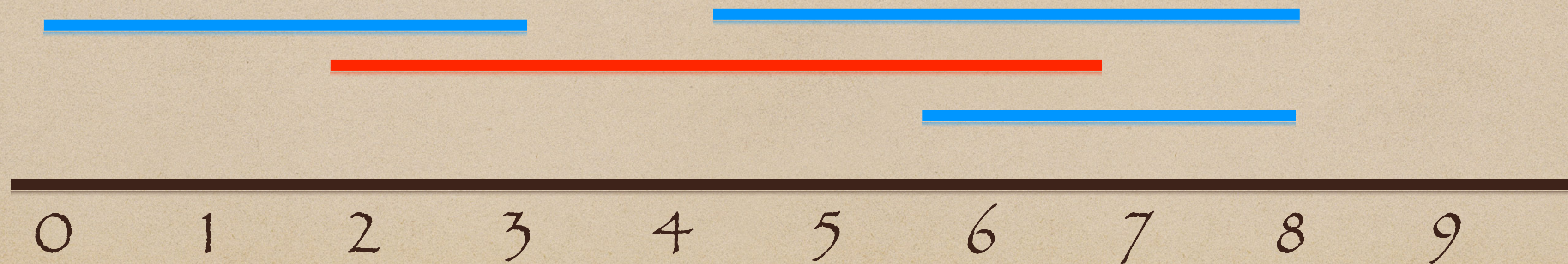
Interval Scheduling - A new Variant

- ◆ Select a set of **disjoint** intervals to maximise the **total length** of the selected intervals.
- ◆ For instance, there could be **profit** proportional to the duration of "use" time.



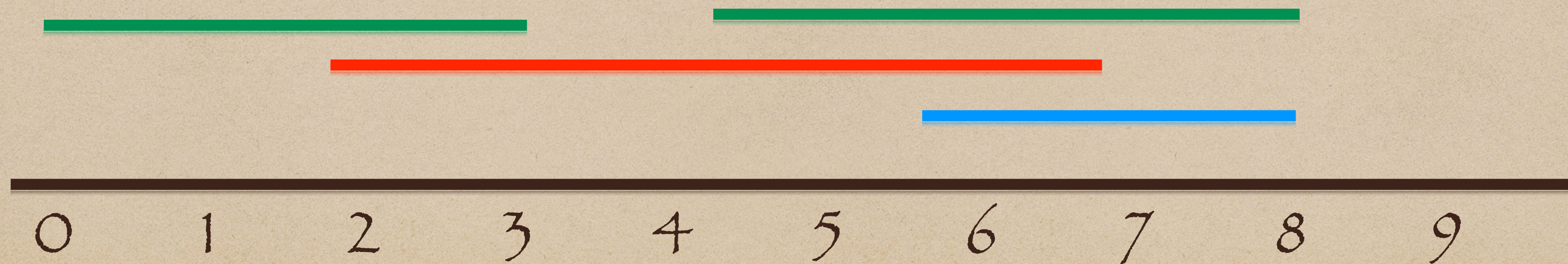
Interval Scheduling - A new Variant

- ◆ Greedy Strategy - 1: Keep picking the **longest** interval



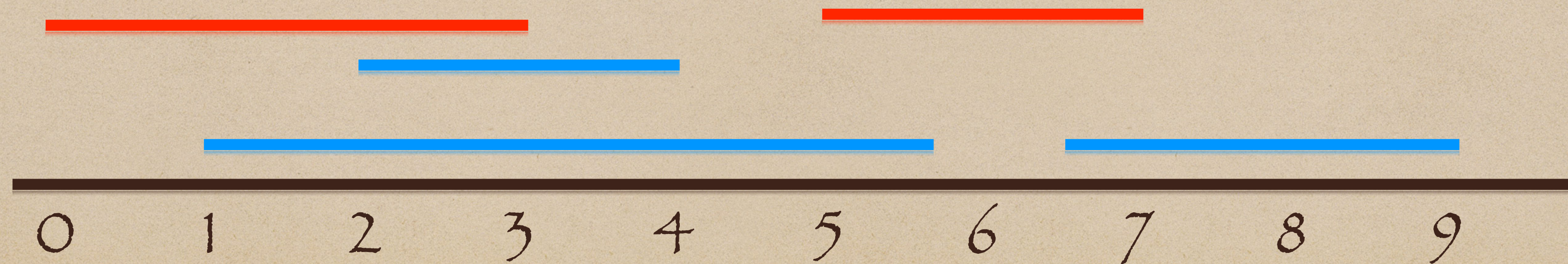
Interval Scheduling - A new Variant

- ◆ Greedy Strategy - 1: Keep picking the **longest** interval



Interval Scheduling - A new Variant

- ◆ Greedy Strategy - II: Choose based on **earliest** starting time



Interval Scheduling - A new Variant

- ◆ Greedy Strategy - II: Choose based on **earliest** starting time



Let's drop Greedy:

Try Dynamic Programming

- ◆ Recall the main mantra: Recursion with better memory management.
- ◆ General idea - Try to **categorise** the possible solutions into **different types**. For each type, find the optimal solution via recursion, and then combine.



Let's drop Greedy:

Try Dynamic Programming

- ◆ Recall the main mantra: Recursion with better memory management.
- ◆ General idea - Try to *categorise* the possible solutions into *different types*. For each type, find the optimal solution via recursion, and then combine.



Or, maybe?



Interval Scheduling with Min. Length Via Dynamic Programming

- ◆ To break into subproblems, we need to create a crack in solution space. This will break the input space into (independent) parts.
- ◆ But, how?

Interval Scheduling with Min. Length Via Dynamic Programming

- ◆ To break into subproblems, we need to create a crack in solution space. This will break the input space into (independent) parts.
- ◆ But, how?
- ◆ Well, we don't know - so, we guess!

Interval Scheduling with Min. Length Via Dynamic Programming



Interval Scheduling with Min. Length Via Dynamic Programming

Q: Is I part of the solution?

If yes, then does it break into two parts remaining set?



Interval Scheduling with Min. Length Via Dynamic Programming

- ◆ Two kinds of solutions.
 - ◆ Either contains I - Case 1.
 - ◆ Or doesn't contain I - Case 2.

Can we find optimal solution for both kinds?

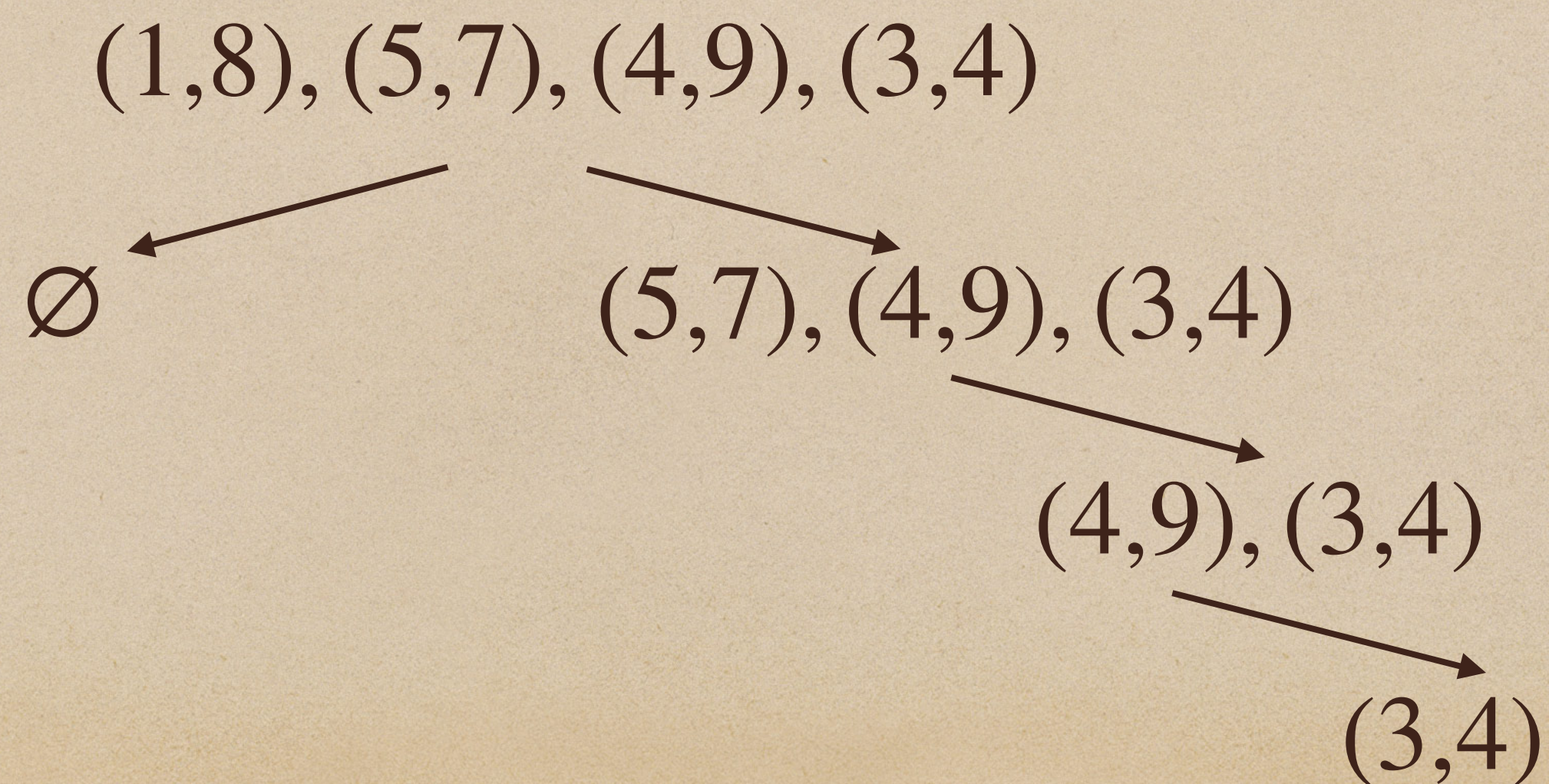
Interval Scheduling with Min. Length Via Dynamic Programming

- ◆ Define, $\mathcal{J}' = \mathcal{J} - \text{Overlap}(I)$
 - ◆ Case 1 (contains I): $OPT(\mathcal{J}') + I$
 - ◆ Define, $\mathcal{J}' = \mathcal{J} - I$
 - ◆ Case 2 (doesn't contains I): $OPT(\mathcal{J}'')$
- $OPT(\mathcal{J}) = \max \{OPT(\mathcal{J}') + I, OPT(\mathcal{J}'')\}$

of Recursive calls

- ♦ $\mathcal{I} = \{(0,5), (1,8), (5,7), (4,9), (3,4)\}$
- ♦ $I_1 = (0,5); \mathcal{I}' = \{(5,7)\}; \mathcal{I}'' = \{(1,8), (5,7), (4,9), (3,4)\}$

$\mathcal{I}$
↓
 $\mathcal{I}'$



of Recursive Calls

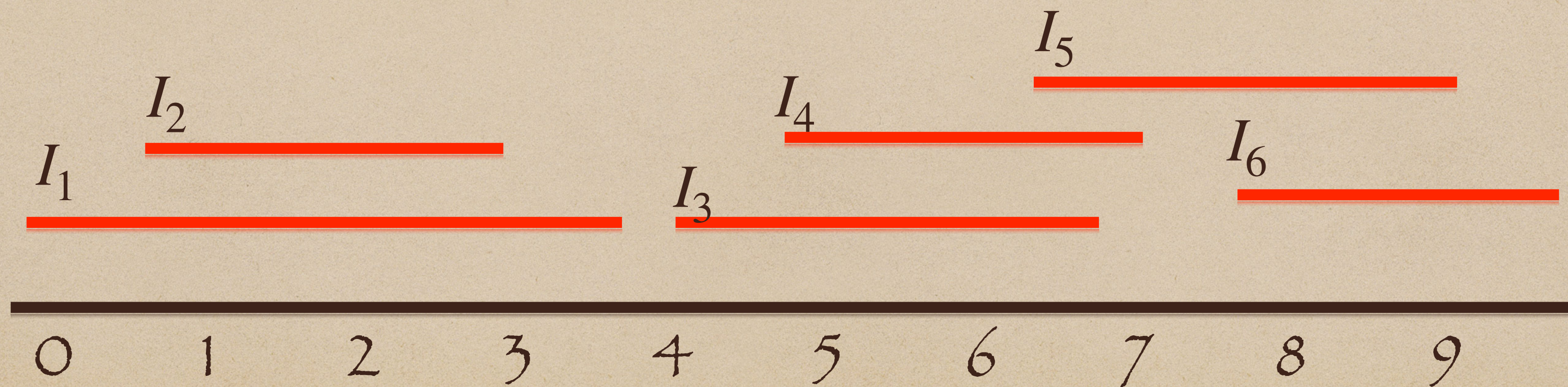
- ◆ Nothing interesting here, we are trying to explore possible solutions path recursively.
- ◆ # of recursive calls seem to grow exponentially, since each call makes two new calls.
- ◆ Efficient implementation is possible, if the # of distinct recursive calls is small.

Less # of Distinct Calls

- ◆ Only $2n$ distinct recursive calls/subproblems.
- ◆ Seems, if the intervals are arranged in a particular order, then the # of distinct subproblems will be small.
- ◆ Possible orders -
 - ◆ Starting time.
 - ◆ Ending time.

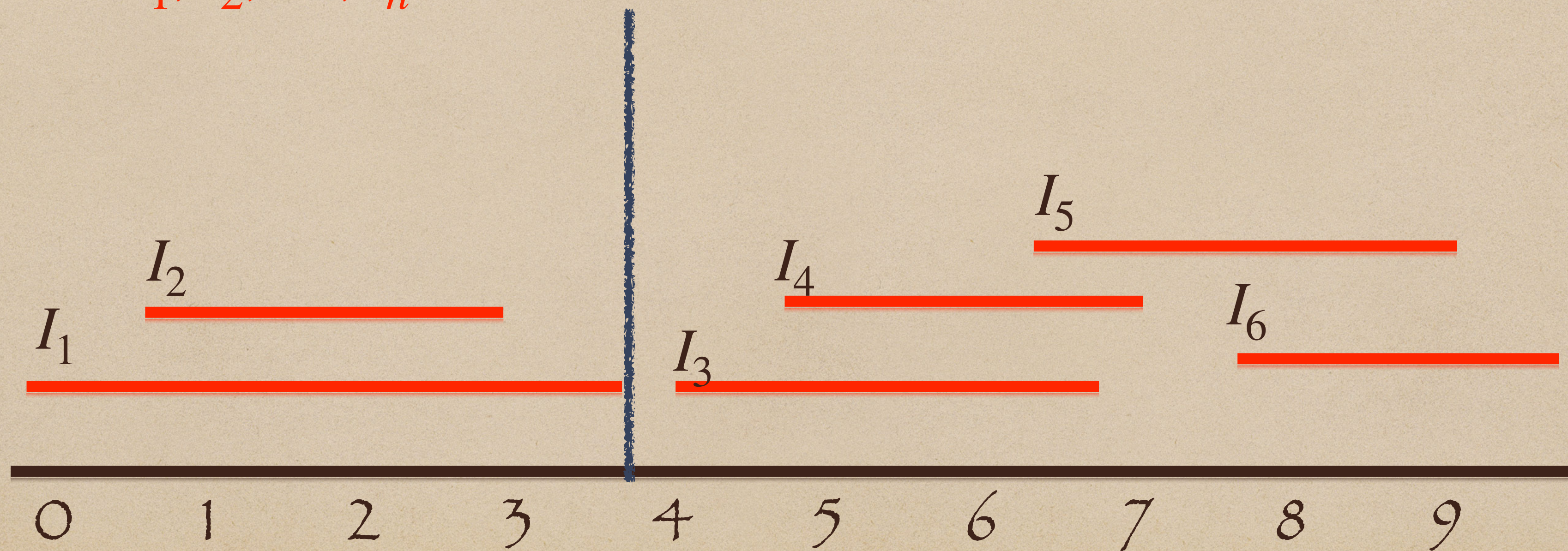
Less # of Distinct Calls

- ♦ Idea: Sort the intervals in the increasing order of the starting time... $I_1, I_2, \dots, I_n$



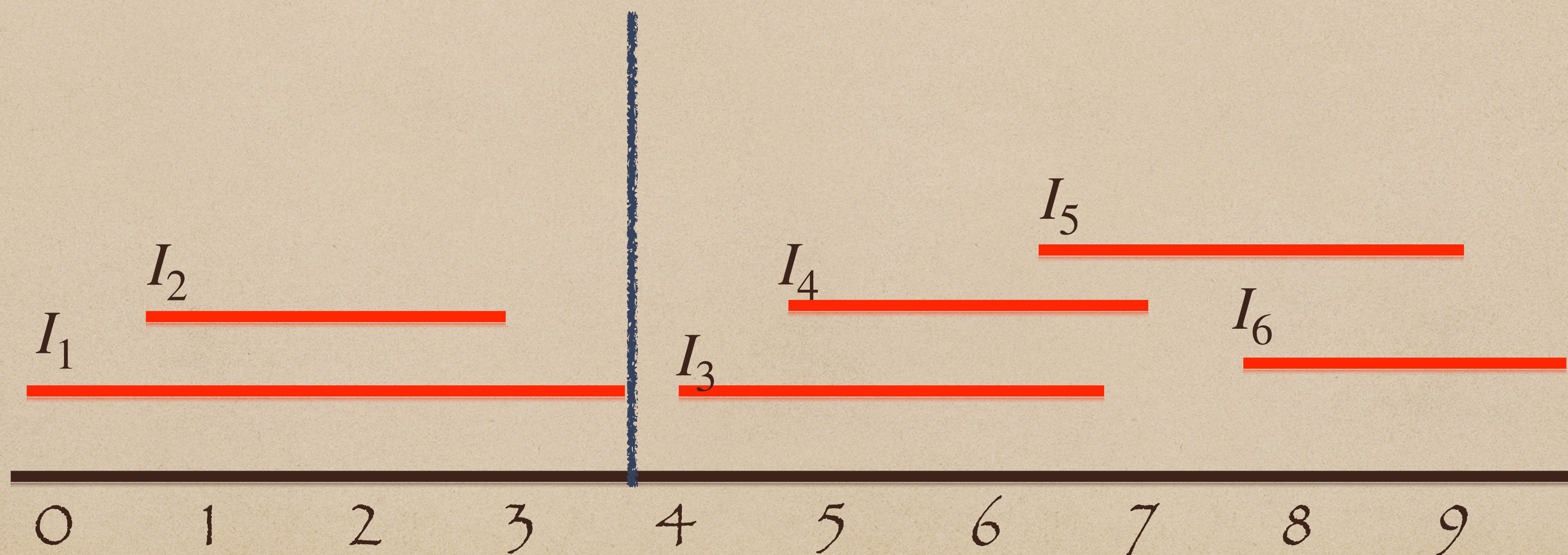
Less # of Distinct Calls

- ♦ Idea: Sort the intervals in the **increasing order of the starting time**... $I_1, I_2, \dots, I_n$



Less # of Distinct Calls

- ◆ Claim: The I/P set of intervals for any recursive calls will look like $\sim \{I_j, I_{j+1}, \dots, I_n\}$ (as opposed to arbitrary set of intervals.)



Less # of Distinct Calls

- ◆ Claim: The I/P set of intervals for any recursive calls will look like $\sim \{I_j, I_{j+1}, \dots, I_n\}$ (as opposed to arbitrary set of intervals.)
- ◆ This happens, because, when we remove intervals overlapping with I_k , the remaining set is simply —
 - ◆ All intervals with starting time $> \text{Endgame}(I_k)$

Less # of Distinct Calls

- ◆ Claim: The I/P set of intervals for any recursive calls will look like $\sim \{I_j, I_{j+1}, \dots, I_n\}$ (as opposed to arbitrary set of intervals.)
- ◆ This happens, because, when we remove intervals overlapping with I_k , the remaining set is simply —
 - ◆ All intervals with starting time $> \text{Endgame}(I_k)$

of distinct subproblems = n

Recursive Implementation

- ♦ $\text{OPT} \leftarrow$ Array with all Zeros
- ♦ $\text{OPT}[n] \leftarrow \text{Length } [I_n]$
- ♦ Output $\text{ALG}[1]$

Recursive Implementation

$\text{ALG}(j)$ computes optimal solution for $\{I_j, I_{j+1}, \dots, I_n\}$

- ♦ $\text{OPT} \leftarrow$ Array with all Zeros

$\text{ALG}(j)$

If $\text{OPT}[j] > 0$, return $\text{OPT}[j]$

- ♦ $\text{OPT}[n] \leftarrow \text{Length}[I_n]$

//means it's already solved and stored.

- ♦ Output $\text{ALG}[1]$

Else,

$\text{OPT}[j] = \max \{ \text{ALG}(j+1), \text{Length}(I_j) + \text{ALG}(p(j)) \}$

Return $\text{OPT}[j]$

Least index k , s.t. $\text{start}(I_k) > \text{end}(I_j)$

Iterative Implementation

- ♦ $\text{OPT} \leftarrow$ Array with all Zeros

- ♦ $\text{OPT}[n] \leftarrow \text{Length}[I_n]$

- ♦ Output $\text{ALG}[1]$

for($j = n - 1$ to 1)

$\text{OPT}[j] \leftarrow \max \{ \text{OPT}[j + 1],$
 $\text{Length}(I_j) + \text{OPT}[p(j)] \}$

Iterative Implementation

- ♦ $\text{OPT} \leftarrow$ Array with all Zeros

for($j = n - 1$ to 1)

- ♦ $\text{OPT}[n] \leftarrow \text{Length}[I_n]$

$\text{OPT}[j] \leftarrow \max \{ \text{OPT}[j + 1],$
 $\text{Length}(I_j) + \text{OPT}[p(j)] \}$

- ♦ Output $\text{ALG}[1]$

Homework: Implement both of these methods to
Understand the difference.

Conclusion

- ◆ Order of processing the input is important to reduce the number of recursive calls.
- ◆ Intervals: Order of starting time/ending time.
- ◆ Sequences: left to right

Try to ensure that the number of subproblems are small.